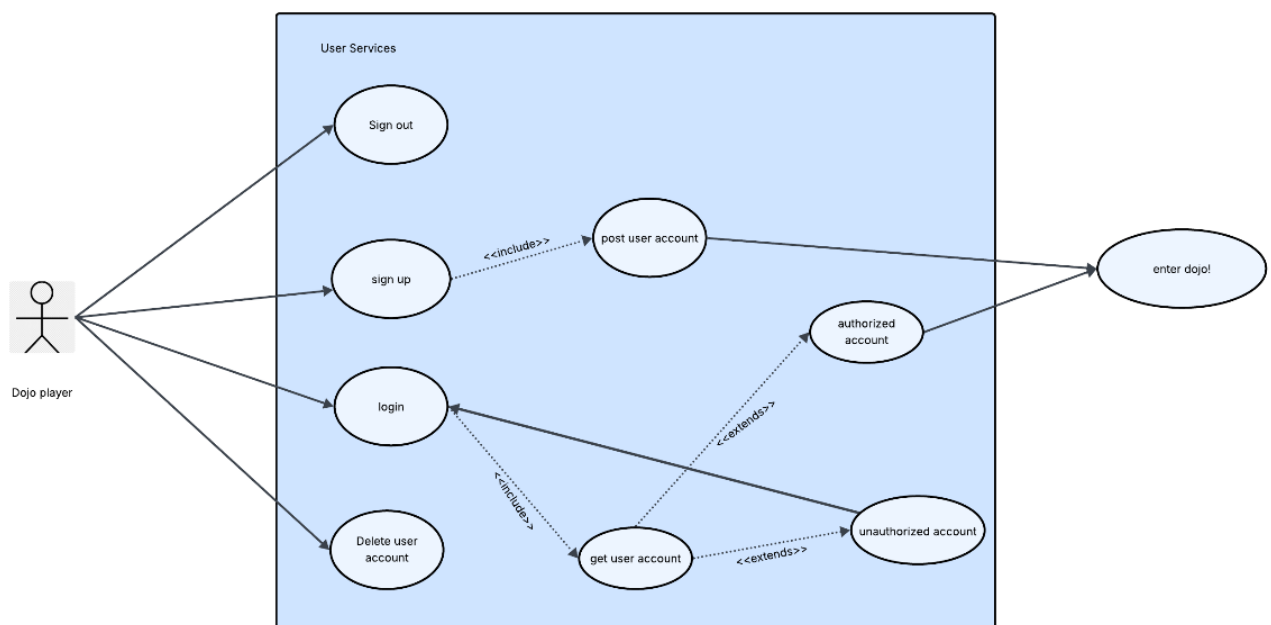


Project Synopsis

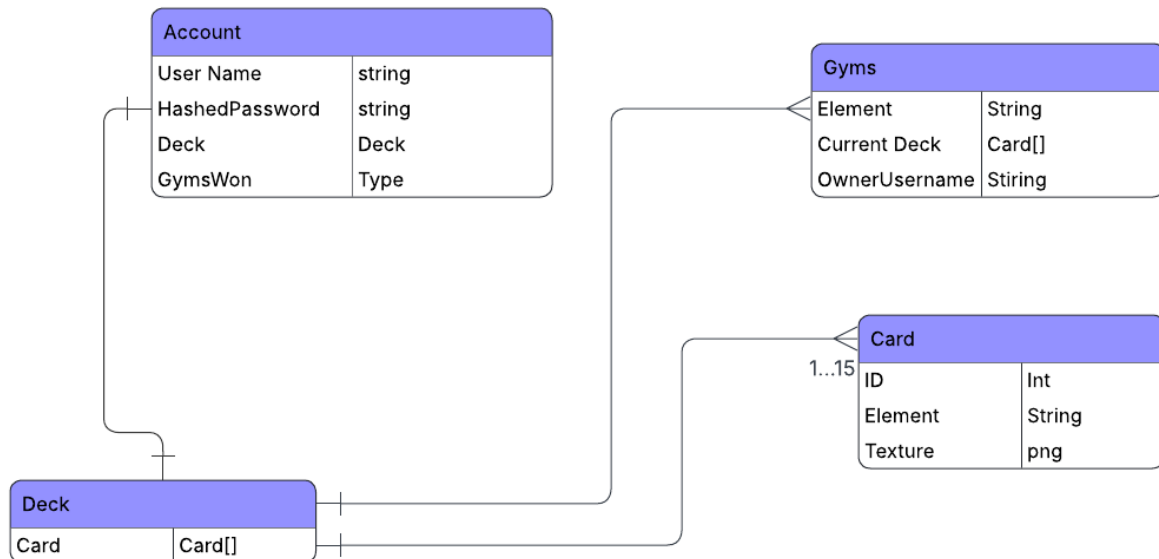
A web-based recreation of Club Penguin's Card Jitsu game featuring five elemental types, strategic card battles, and AI gameplay with user accounts and decks.

Login Page Structure Diagram



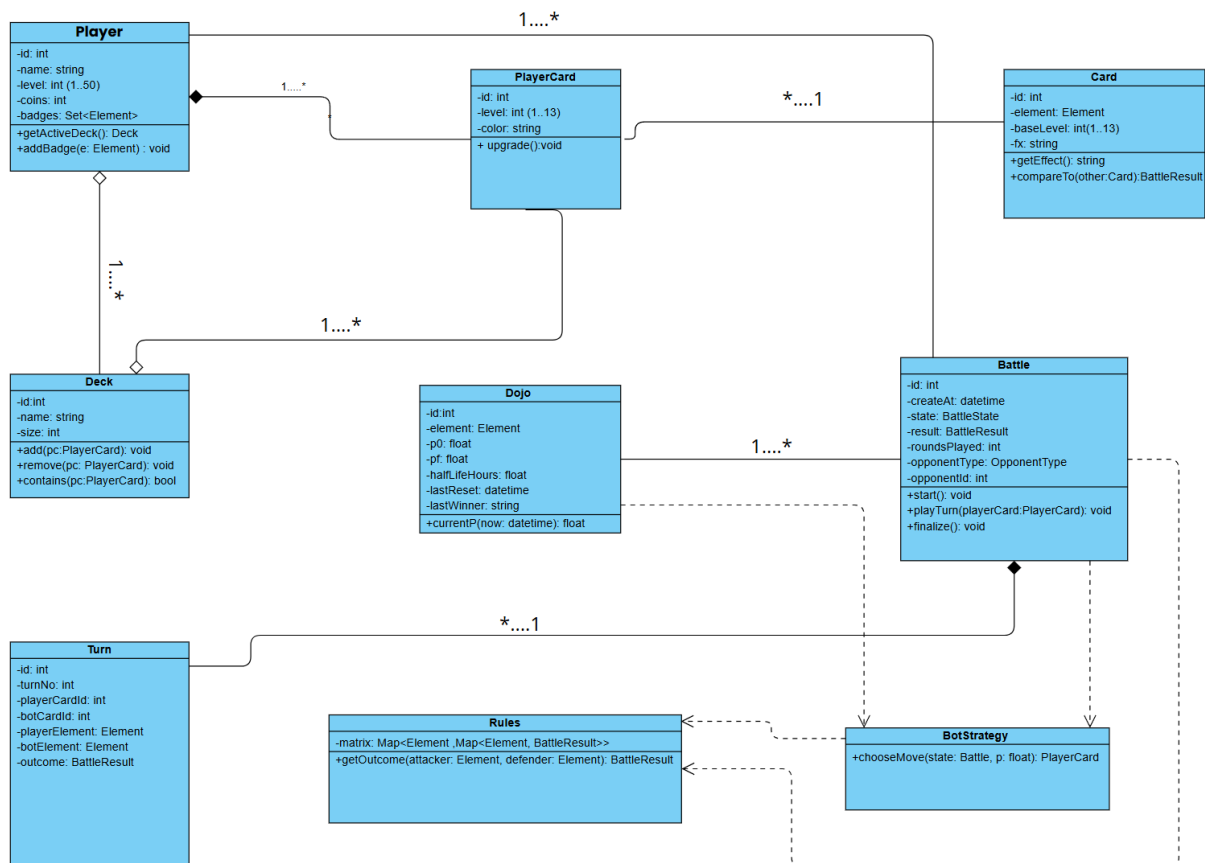
This UML case diagram shows the progress flow on the dojo player and the system login service. Each user will have a unique username and password that they will have to keep track of their progress. There are multiple cases to get past the login screen, which may include multiple states such as the user logging in to their account or creating one if they don't already have one. From there, there are <<include>> and <<extend>> scenarios including invalidity if they type in a wrong user/password or if they enter without an account, which will prompt them to reenter a valid user credential. After going through the user services screen depending on if they were a new or returning user, they will be able to enter the dojo with their information to start or continue playing. The user will can encounter the “delete account” function if they choose to start over from the beginning.

Data Structure Diagram



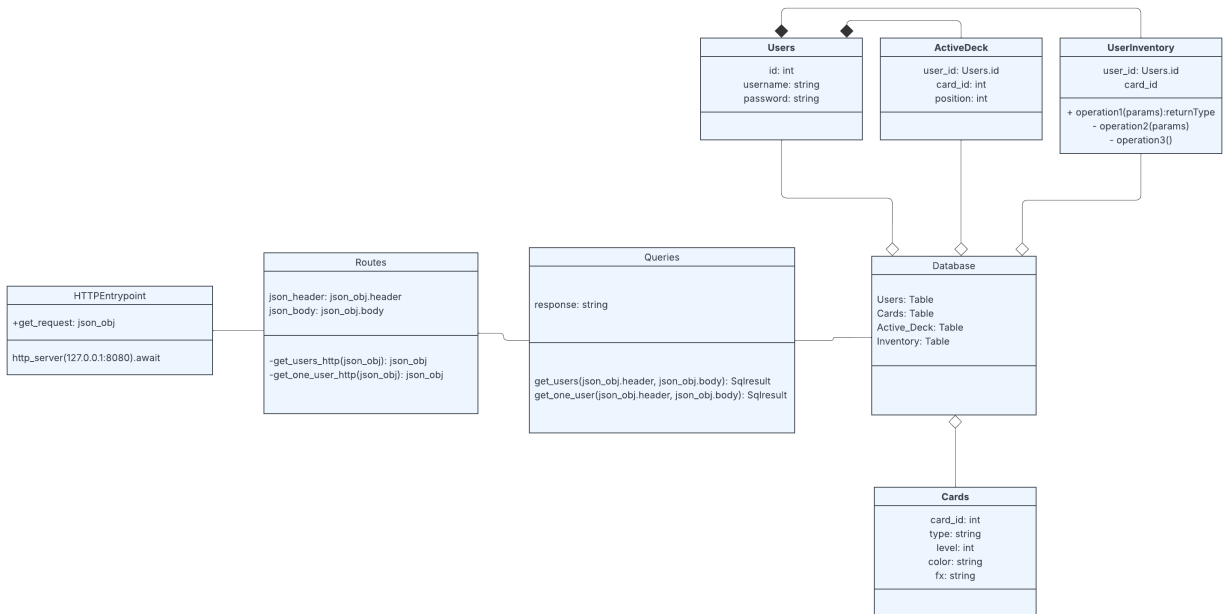
This diagram shows the entities and relationships between them as they exist in our database and throughout the backend structure. They also communicate the shape of the models that will be passed using JSON over network requests. The main entity that will be used and passed is the account entity. Every user will have an account row in our database. This will give security details but will also be how their deck is tracked and what gyms they have won. This will be helpful in displaying this information to any user that has won a gym. The gyms, however, will not directly link back the to the account except that the user's username will be saved with gym upon a user winning that gym. The gyms data will be displayed on the map.

Frontend Data Path Diagram



This activity diagram illustrates the dynamic flow of the Dodo Dojo game loop during a battle sequence. The process begins with battle initialization, where player and bot decks are loaded and the Rules engine is prepared. Each cycle represents a turn: the player selects a card, and the bot chooses its move through the BotStrategy interface using probability-based logic. Both cards are then evaluated by the Rules component to determine the outcome of the turn. Scores and game states are updated accordingly, looping until a win condition is met. The diagram emphasizes system behavior over time, highlighting how control flows between the player, battle engine, bot AI, and rules evaluation to sustain continuous gameplay.

Backend Request Path Diagram



This diagram shows the relationship between the structure that makes up the http server and the database that holds all user and game data. Starting at the HTTP Entry point, this is where when a connection is made, a json_obj is the incoming request that then gets passed to the routes class. In the routes class, depending on the http request in the header, it can call the correct function that will pass off the data to the queries class. In the queries class this is where the sql query is ran with the data that was passed from the routes functions. The query goes to the database where there is other classes that represent the different tables that form the entire database.